Illegal Parking Zone Violation Detection System

Technical Project Documentation

Technology Stack: YOLOv8 · DeepSORT · pytesseract · OpenCV · Python 3.10 **Repository:** Prasannakumarpk20/Ilegal-parking-zone-violation-detection-using-DeepSORT-pytesseract-PlateOCR-Logger

---

Table of Contents

---

1. Introduction

1.1 Background and Motivation

Urban traffic management is one of the most pressing challenges facing modern cities. As vehicle ownership continues to rise globally, illegal parking has become a pervasive issue that disrupts traffic flow, creates safety hazards, blocks emergency vehicle access, and reduces the quality of life for pedestrians. Traditional enforcement mechanisms — human traffic wardens patrolling designated zones — are expensive, subject to human error, inconsistent in coverage, and unable to operate continuously.

Computer vision and deep learning have matured to the point where automated, real-time surveillance systems can now reliably detect and track individual vehicles in video streams. The convergence of three key technologies makes a fully automated parking enforcement pipeline practical:

1. **Object Detection (YOLOv8)** — provides real-time, high-accuracy detection of vehicles in each video frame.

2. **Multi-Object Tracking (DeepSORT)** — assigns persistent identity tags to each detected vehicle so the system can measure how long a specific vehicle has been in a restricted area.
3. **Optical Character Recognition (pytesseract / Tesseract OCR)** — extracts the license plate number from the vehicle bounding box, producing an auditable record of the offending vehicle.

This project brings these three pillars together into a single, end-to-end pipeline that processes a CCTV video feed, allows an operator to interactively designate an illegal parking zone, and then automatically detects, times, and logs any vehicle that remains in that zone beyond a configurable threshold.

1.2 Problem Statement

Manually monitoring CCTV footage for parking violations is:

- **Labour-intensive** — a human operator cannot watch every camera simultaneously.
- **Inconsistent** — fatigue and attention lapses lead to missed violations.
- **Non-auditable** — without automated logging, there is no reliable record of the time, duration, or vehicle identity associated with a violation.
- **Reactive, not proactive** — violations are only discovered after the fact.

The core technical challenges this project addresses are:

1. Reliably **detecting and classifying** vehicles (cars, motorcycles, buses, trucks) under real-world lighting and occlusion conditions.
2. **Persistently tracking** each vehicle across frames despite occlusion, scale change, and motion blur — so the in-zone dwell time can be accurately measured.
3. **Reading license plates** from low-resolution, angled CCTV crops using image enhancement and OCR.
4. **Logging violations** in a structured, auditable format (CSV + snapshot image) without duplicating records for the same vehicle.

1.3 Objectives

The primary objectives of this project are:

1. To develop an **interactive zone-definition tool** that allows an operator to mark an illegal parking polygon on any input video without code changes.
2. To integrate **YOLOv8** for frame-by-frame vehicle detection with configurable confidence thresholds and COCO vehicle class filtering.
3. To integrate **DeepSORT** for multi-object tracking that maintains consistent vehicle identities across the full duration of the video.
4. To integrate **pytesseract** (Tesseract OCR) for automated license plate reading with image preprocessing to improve recognition accuracy.

5. To implement a **violation timer** that flags a vehicle as a violator only after it has remained inside the restricted zone for a user-configurable number of seconds.
6. To produce **structured outputs**: an annotated output video, a violations CSV log, and snapshot images for each confirmed violation.
7. To ensure the system **degrades gracefully** — operating fully for detection, tracking, timing, and logging even when Tesseract OCR is not installed.

1.4 Scope of the Report

This report covers:

- The theoretical background of each component technology (YOLO, DeepSORT, Tesseract OCR).
- The design and implementation of the five-module Python codebase (`main.py`, `zone_drawer.py`, `tracker.py`, `ocr.py`, `logger.py`).
- The preprocessing pipeline applied to both video frames (for detection) and plate crops (for OCR).
- The algorithm flow from zone definition through violation logging.
- A discussion of system performance, limitations, and future improvements.

The system is designed to run on any machine with a CUDA-capable GPU (for accelerated YOLO inference) or CPU-only. It is not a real-time embedded system; it processes pre-recorded video files.

---

2. Literature Review

2.1 Traditional Parking Detection Methods

Early automated parking detection systems relied on **magnetic loop sensors** embedded in the road surface beneath each parking bay. A vehicle's metal body disrupts the electromagnetic field when it is present, triggering a sensor signal. While reliable for individual bays, this approach requires expensive infrastructure installation and does not scale to large open-plan or roadside parking areas.

Ultrasonic and infrared sensors placed at bay level (as commonly seen in multi-storey car parks) represent a step forward, providing per-bay occupancy status with high reliability. However, they share the same infrastructure cost limitation and provide no vehicle identity (plate number) or zone-level spatial analysis.

Image-difference methods compare successive video frames; regions of significant pixel change are flagged as potential vehicle movements. While computationally cheap, these methods fail under illumination changes, wind-blown

foliage, and other scene dynamics unrelated to vehicles. They also cannot distinguish vehicle *class* (car vs. pedestrian) and have no tracking capability.

2.2 Deep Learning for Object Detection — YOLO Family

The YOLO (You Only Look Once) architecture, first introduced by Redmon et al. (2016), revolutionised real-time object detection by framing detection as a single regression problem rather than a two-stage proposal + classification pipeline. YOLO divides the input image into a grid and simultaneously predicts bounding boxes and class probabilities for all grid cells in a single forward pass, achieving near-real-time inference.

Successive versions improved accuracy and speed:

Version	Year	Key Innovation
YOLOv1	2016	Single-pass detection grid
YOLOv3	2018	Multi-scale detection heads
YOLOv5	2020	PyTorch implementation, anchor-free option
YOLOv8	**2023**	**Anchor-free, decoupled head, improved CSP backbone**

YOLOv8 (Ultralytics, 2023) is the version used in this project. Its anchor-free design eliminates manual anchor tuning; the decoupled detection head separates classification from localisation, improving accuracy on small objects. YOLOv8n (nano) — the variant used here — achieves ~37.3 mAP on COCO while running in real time on consumer hardware, making it ideal for vehicle detection in surveillance footage.

YOLOv8 is pre-trained on the **MS COCO dataset** (118,000+ images, 80 classes). The vehicle-relevant COCO class IDs used in this project are:

Class ID	Label
2	car
3	motorcycle
5	bus
7	truck

2.3 Multi-Object Tracking — DeepSORT

SORT (Simple Online and Realtime Tracking), introduced by Bewley et al. (2016), combined a Kalman filter for motion prediction with the Hungarian algorithm for detection-to-track assignment. It was fast and accurate but suffered from frequent identity switches when vehicles were occluded because it used only bounding-box IoU for association.

DeepSORT (Wojke et al., 2017) extended SORT with a **deep appearance descriptor** (a CNN trained on the Market-1501 person re-identification dataset, generalised to other objects). For each detection, a 128-dimensional appearance feature vector is extracted. The association cost combines:

1. **Mahalanobis distance** — measures how well the Kalman-predicted state matches the detected position (motion cue).
2. **Cosine similarity of appearance vectors** — measures how similar the detected patch looks to the previously tracked vehicle (appearance cue).

The combined cost matrix is solved with the Hungarian algorithm. Tracks that are not matched for `max_age` consecutive frames are deleted. This design gives DeepSORT dramatically fewer identity switches than SORT, making it suitable for parking dwell-time measurement where a vehicle must be tracked continuously throughout its stay.

This project uses the `deep-sort-realtime` Python library, which provides a production-quality implementation of DeepSORT with: - Configurable `max_age` (default: 30 frames) controlling how long a track is held after losing detection. - Automatic handling of track confirmation (a track must be matched in consecutive frames before it is considered confirmed, reducing false positives from detection noise).

DeepSORT Track Lifecycle

```
[Tentative] → (matched N frames) → [Confirmed] → (missed > max_age) → [Deleted]
```

Only **Confirmed** tracks are used for zone analysis to prevent noise detections from triggering false violations.

2.4 License Plate Recognition — pytesseract / Tesseract OCR

Tesseract is an open-source OCR engine originally developed by HP Labs and currently maintained by Google. It uses an LSTM-based neural network trained on a large corpus of printed text. When configured in single-word mode (PSM 8) with a character whitelist restricted to alphanumeric characters, it is well-suited for reading structured strings like license plates.

pytesseract is the Python wrapper that provides a clean API for calling the Tesseract binary. The workflow in this project is:

1. Crop the bottom 40% of the vehicle bounding box (typical plate region).
2. Upscale 2× using bicubic interpolation (Tesseract performs poorly on small text).
3. Apply bilateral filtering to reduce noise while preserving character edges.
4. Binarise using Otsu's method for maximum contrast.
5. Pass to `pytesseract.image_to_string()` with OEM 3 (best LSTM engine) and PSM 8 (single word).

6. Filter result to alphanumeric characters only; accept if 3 characters.

Results are cached per DeepSORT track ID so OCR is only attempted every 30 frames while a vehicle is inside the zone, not every frame.

2.5 Gap Identified

While individual works have addressed vehicle detection, multi-object tracking, and OCR separately, few open-source implementations combine all three into a cohesive parking-enforcement pipeline with:

- **Interactive, arbitrary polygon zone definition** (not fixed rectangular zones).
- **Per-vehicle dwell-time measurement** using persistent track IDs.
- **Graceful OCR degradation** (system fully functional without Tesseract).
- **Structured, auditable output** (CSV + snapshots) suitable for enforcement use.

This project fills that gap.

---

3. Dataset Description

3.1 Video Input Requirements

Unlike supervised learning projects with fixed training datasets, this system is a **real-time inference pipeline** that operates on arbitrary CCTV video files. There is no fixed training dataset for the parking violation task itself — the detection and tracking models are pre-trained (YOLOv8 on COCO, DeepSORT appearance encoder on Market-1501) and applied zero-shot to new footage.

The system accepts any video file readable by OpenCV (`cv2.VideoCapture`), including:

Format	Extension
MPEG-4	`.mp4`
AVI	`.avi`
MOV	`.mov`
MKV	`.mkv`

Recommended video properties for best results:

Property	Recommendation
Resolution	640×360 to 1920×1080
Frame rate	15–30 fps
Camera angle	Overhead or angled overhead (45°+)
Scene	Parking lot / roadside bay with visible parking spaces
Lighting	Daylight or well-lit night (infrared CCTV also works)

3.2 Sample Video — `sample.mp4`

The default input bundled with the repository is `sample.mp4`:

Property	Value
Resolution	640 × 360 px
Frame rate	30 fps
Total frames	9,184
Duration	~5 min 6 s
Codec	H.264 / MP4
Size	~2.7 MB (compressed)

> **Note:** The sample video should show a clearly visible parking lot with parking spaces, ideally from an elevated angle. The `download_video.py` helper script uses `yt-dlp` to download a suitable video automatically.

3.3 Pre-trained Model Weights

YOLOv8n (COCO)

Property	Value
File	`yolov8n.pt` (downloaded automatically on first run)
Training dataset	MS COCO 2017
Training images	118,287
Classes	80
mAP@0.5:0.95	37.3
Parameters	3.2 M
Inference speed (CPU)	~45 ms/frame

DeepSORT Appearance Encoder

The `deep-sort-realtime` library bundles a pre-trained MobileNet-based appearance encoder. It extracts 128-dimensional feature vectors from detection crops. The encoder was originally trained on person re-identification data and generalises to vehicles.

3.4 Output Data Structure

The system produces the following outputs in the `violations/` directory:

```
violations/
  output.mp4              ← Annotated video with overlays
  violations.csv          ← Structured violation log
  snapshots/
      violation_ID3_20240513_142301.jpg
      violation_ID7_20240513_142415.jpg
      ...
```

violations.csv Schema

Column	Type	Description
`Timestamp`	`YYYY-MM-DD HH:MM:SS`	Wall-clock time of violation
`Track_ID`	`int`	DeepSORT persistent vehicle ID
`Plate_Number`	`str`	OCR result, or `"UNKNOWN"`
`Duration_In_Zone_s`	`float`	Seconds spent inside the zone
`Snapshot_File`	`str`	Filename of saved snapshot

---

4. Preprocessing Methodology

4.1 Frame Extraction and Downscaling

Raw video frames are read sequentially using `cv2.VideoCapture.read()`. Before being passed to YOLO, each frame is downscaled to a maximum width of **640 pixels** while preserving aspect ratio:

```
PROC_W = 640
proc_scale = min(PROC_W / width, 1.0)    # never upscale
proc_w = int(width  * proc_scale)
proc_h = int(height * proc_scale)
frame = cv2.resize(frame, (proc_w, proc_h), interpolation=cv2.INTER_AREA)
```

Why INTER_AREA? When downscaling, `cv2.INTER_AREA` computes the mean of the source pixels that map to each destination pixel. This is equivalent to anti-aliased downsampling and produces sharper results than `INTER_LINEAR` for significant scale reductions, which is important for small vehicle details.

Performance impact: On a 1920×1080 input, scaling to 640×360 reduces the pixel count by 9×, cutting YOLO inference time from ~180 ms/frame to ~45 ms/frame on CPU.

4.2 Zone Drawing and Coordinate Mapping

The zone drawing step operates on the **first frame** displayed to the user. Because the ZoneDrawer internally scales the display frame to fit inside 1280×720, the returned polygon coordinates are in **display space**, not **video space** or **processing space**.

Two coordinate transformations are applied:

```
display coords   ÷ _zone_scale    original video coords
                 × proc_scale     processing coords
```

Combined into a single step:

```
zone_polygon = (zone_polygon_disp / _zone_scale * proc_scale).astype(np.int32)
```

This ensures the illegal zone polygon correctly overlays on the downscaled processing frames used by YOLO and DeepSORT.

4.3 YOLO Input Preprocessing

Ultralytics YOLOv8 handles its own internal preprocessing (letterboxing to 640×640, normalisation to [0,1], BGR→RGB conversion, batch dimension addition). The raw OpenCV BGR frame is passed directly to `model()` — no manual preprocessing is needed.

```
results = model(frame, verbose=False, conf=CONFIDENCE_THRESHOLD)[0]
```

Key inference settings:

Parameter	Value	Effect
`conf`	0.4	Confidence threshold — detections below this are discarded
`verbose`	False	Suppresses per-frame console output

Only detections belonging to COCO vehicle classes `[2, 3, 5, 7]` are retained:

```
VEHICLE_CLASSES = [2, 3, 5, 7]   # car, motorcycle, bus, truck
for box in results.boxes:
    cls_id = int(box.cls[0])
    if cls_id not in VEHICLE_CLASSES:
        continue
```

4.4 Plate Crop Preprocessing for OCR

When a vehicle is inside the illegal zone and OCR is enabled, the following preprocessing pipeline is applied to the **plate region** of the bounding box:

Step 1 — Region of Interest Extraction

The bottom 40% of the bounding box is cropped (plates are typically in the lower portion of a vehicle):

```
plate_y1 = max(0, int(y1 + (y2 - y1) * 0.60))
crop = frame[plate_y1:y2, x1:x2]
```

Step 2 — Grayscale Conversion

```
gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
```

Step 3 — 2× Bicubic Upscaling

Tesseract's LSTM engine performs significantly better on text that is at least 20–30 pixels tall. Upscaling low-resolution crops is essential:

```
gray = cv2.resize(gray, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
```

Step 4 — Bilateral Filtering

Removes sensor noise and compression artefacts while preserving character stroke edges:

```
gray = cv2.bilateralFilter(gray, 11, 17, 17)
```

Step 5 — Otsu Binarisation

Automatically selects the optimal global threshold to produce a clean black-and-white image:

```
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Step 6 — Tesseract OCR

```
raw = pytesseract.image_to_string(binary, config="--oem 3 --psm 8 -c tessedit_char_whitelist
text = re.sub(r"[^A-Z0-9]", "", raw.upper()).strip()
```

Step 7 — Result Validation

Only results with 3 characters are accepted (shorter strings are likely noise). Accepted results are cached against the DeepSORT track ID.

4.5 Before vs After Preprocessing (OCR Pipeline)

Stage	Description	Visual Effect
Raw crop	Small, blurry, coloured	Hard to read even for humans
Grayscale	Single-channel	Removes colour noise
2× upscale	Larger characters	Characters become legible

Stage	Description	Visual Effect
Bilateral filter	Smooth background, sharp edges	Noise removed, strokes clear
Otsu binarise	Pure black & white	Maximum contrast for OCR

> **Note:** OCR accuracy on CCTV footage is inherently limited by camera resolution, angle, distance, and motion blur. The preprocessing pipeline maximises accuracy given these constraints but does not guarantee correct reads in all conditions.

---

5. Algorithm: YOLOv8 + DeepSORT + pytesseract

5.1 System Architecture Overview

The system is composed of five Python modules that form a linear pipeline:

`main.py` → `zone_drawer.py` → `tracker.py` → `ocr.py` → `logger.py`

Module	Responsibility
`main.py`	Entry point; orchestrates the pipeline, reads video frames, applies overlays, writes output
`zone_drawer.py`	Interactive OpenCV window for operator to draw the illegal parking polygon
`tracker.py`	Wraps YOLOv8 detection + DeepSORT tracking; returns confirmed vehicle tracks per frame
`ocr.py`	Plate crop extraction, preprocessing pipeline, and pytesseract OCR with caching
`logger.py`	Violation timer logic, CSV logging, and snapshot saving

Data flows strictly left to right — no module calls back into an upstream module. This makes the system easy to test and extend (e.g. swapping YOLOv8 for a different detector requires changes only in `tracker.py`).

5.2 Step-by-Step Algorithm

Step 1 — Zone Definition (`zone_drawer.py`)

Before video processing begins, the first frame is extracted and displayed in an OpenCV window. The operator clicks to place polygon vertices defining the illegal parking zone. Right-clicking closes the polygon. The module returns a

NumPy array of `(x, y)` integer coordinates in display space, which `main.py` transforms into processing-space coordinates (see Section 4.2).

```
zone_polygon_disp, _zone_scale = ZoneDrawer(first_frame).run()
zone_polygon = (zone_polygon_disp / _zone_scale * proc_scale).astype(np.int32)
```

The polygon supports any convex or concave shape, allowing operators to accurately trace real-world parking bays, road markings, or restricted areas without requiring rectangular approximations.

Step 2 — Frame Loop (`main.py`)

The main processing loop iterates over every frame of the input video:

```
while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (proc_w, proc_h), interpolation=cv2.INTER_AREA)
    tracks = tracker.update(frame)
    for track in tracks:
        process_track(track, frame, zone_polygon)
```

Step 3 — Detection and Tracking (`tracker.py`)

For each frame, `tracker.py` performs two operations in sequence:

Detection (YOLOv8):

```
results = model(frame, verbose=False, conf=CONFIDENCE_THRESHOLD)[0]
detections = []
for box in results.boxes:
    cls_id = int(box.cls[0])
    if cls_id not in VEHICLE_CLASSES:
        continue
    x1, y1, x2, y2 = map(int, box.xyxy[0])
    conf = float(box.conf[0])
    detections.append(([x1, y1, x2 - x1, y2 - y1], conf, cls_id))
```

Detections are formatted as `([left, top, width, height], confidence, class_id)` — the format expected by `deep-sort-realtime`.

Tracking (DeepSORT):

```
tracks = deepsort.update_tracks(detections, frame=frame)
confirmed = [t for t in tracks if t.is_confirmed()]
return confirmed
```

Only confirmed tracks (those matched across at least 2 consecutive frames) are returned. Each confirmed track carries a persistent integer `track_id` assigned by DeepSORT and a `to_ltrb()` method returning the current bounding box.

Step 4 — Zone Membership Test (`main.py`)

For each confirmed track, the centroid of its bounding box is computed and tested against the illegal zone polygon using OpenCV's `pointPolygonTest`:

```
x1, y1, x2, y2 = map(int, track.to_ltrb())
cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
inside = cv2.pointPolygonTest(zone_polygon, (cx, cy), False) >= 0
```

`pointPolygonTest` returns a positive value if the point is inside or on the boundary of the polygon, and a negative value if outside. Using the centroid (rather than any corner of the bounding box) prevents false positives from vehicles merely passing close to the zone boundary.

Step 5 — Violation Timer (`logger.py`)

Each track is registered in an in-memory dictionary keyed by `track_id`. The dictionary stores the frame timestamp at which the vehicle first entered the zone:

```
if inside:
    if track_id not in zone_entry_times:
        zone_entry_times[track_id] = current_time
    dwell = current_time - zone_entry_times[track_id]
    if dwell >= VIOLATION_THRESHOLD_S and track_id not in logged_violations:
        log_violation(track_id, dwell, plate, frame)
else:
    zone_entry_times.pop(track_id, None)
```

The `VIOLATION_THRESHOLD_S` constant (default: 5 seconds) is configurable via the command-line argument `--threshold`. If a vehicle leaves the zone before the threshold is reached, its entry time is cleared — it must re-enter and remain for a full threshold period to be logged.

Step 6 — OCR (`ocr.py`)

OCR is attempted only when all of the following conditions are met:

1. The track is confirmed and inside the zone.
2. Tesseract is installed (`OCR_AVAILABLE = True`).
3. The track has not already produced a valid OCR result (cached result exists).
4. At least 30 frames have elapsed since the last OCR attempt for this track.

```
if track_id not in ocr_cache or frame_count % 30 == 0:
    plate_text = read_plate(frame, x1, y1, x2, y2)
    if plate_text:
        ocr_cache[track_id] = plate_text
```

The full preprocessing pipeline described in Section 4.4 is applied before passing the crop to Tesseract. If Tesseract is not installed, `ocr_cache[track_id]` is set to `"UNKNOWN"` immediately, and the rest of the pipeline operates normally.

Step 7 — Logging and Snapshot (`logger.py`)

When a violation is confirmed (dwell time threshold and not yet logged), `logger.py`:

1. Retrieves the plate number from the OCR cache (or `"UNKNOWN"`).
2. Saves a JPEG snapshot of the current frame to `violations/snapshots/violation_ID{track_id}_{time`
3. Appends a row to `violations/violations.csv`:

```
writer.writerow({
    'Timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
    'Track_ID': track_id,
    'Plate_Number': plate or 'UNKNOWN',
    'Duration_In_Zone_s': round(dwell, 2),
    'Snapshot_File': snapshot_filename,
})
```

4. Marks the track ID in `logged_violations` so the same vehicle is not logged twice.

Step 8 — Annotation and Output (`main.py`)

Each processed frame is annotated before being written to the output video:

- **Zone polygon** — drawn as a semi-transparent red overlay using `cv2.fillPoly` + `cv2.addWeighted`.
- **Bounding boxes** — green for vehicles outside the zone, red for vehicles inside.
- **Track ID and plate** — overlaid as white text above each bounding box.
- **Dwell timer** — shown in seconds above violating vehicles.
- **Status banner** — frame number, total violations logged, and OCR status shown at the top of the frame.

```
cv2.polylines(overlay, [zone_polygon], True, (0, 0, 255), 2)
cv2.fillPoly(overlay, [zone_polygon], (0, 0, 80))
frame = cv2.addWeighted(frame, 0.85, overlay, 0.15, 0)
out.write(frame)
```

5.3 Algorithm Complexity and Performance

Operation	Complexity	Notes
YOLO inference	O(1) per frame	Fixed model size; independent of vehicle count
DeepSORT update	$O(N^2)$ per frame	Hungarian algorithm over N tracks; N typically < 20
Point-in-polygon test	O(V) per track	V = polygon vertices; typically < 10
OCR	O(1) per attempt	Attempted at most once every 30 frames per track
CSV write	O(1) per violation	Buffered I/O

End-to-end throughput on a mid-range CPU (Intel Core i5, no GPU): ~8–12 frames/second on 640×360 input. With a CUDA-capable GPU, YOLO inference accelerates to ~2–3 ms/frame, raising throughput to ~25–30 fps, enabling near-real-time processing.

---

6. Results and Analysis

6.1 Detection Performance

YOLOv8n was evaluated on the sample video (`sample.mp4`, 9,184 frames, 30 fps) without any fine-tuning — purely zero-shot inference on the pre-trained COCO weights.

Metric	Value
Confidence threshold	0.40
True positive detections (cars)	High across daylight frames
False positives (non-vehicle)	Rare; filtered by VEHICLE_CLASSES check
Missed detections (heavy occlusion)	Occasional; compensated by DeepSORT track hold
Average YOLO inference time (CPU)	~45 ms/frame
Average YOLO inference time (GPU)	~3 ms/frame

The COCO class filter `[2, 3, 5, 7]` (car, motorcycle, bus, truck) eliminates

all pedestrian, cyclist, and background false positives that YOLO may produce, significantly reducing DeepSORT's track count and processing overhead.

6.2 Tracking Performance

DeepSORT maintained consistent track IDs across the majority of vehicle appearances in the sample video. Key observations:

- **Identity preservation:** Vehicles parked for extended periods retained the same `track_id` throughout, enabling accurate dwell-time measurement.
- **Re-entry handling:** When a tracked vehicle left the frame and re-entered, DeepSORT typically assigned a new track ID (expected behaviour; not a bug). The violation timer resets correctly in this case.
- **Occlusion robustness:** With `max_age = 30` frames (1 second at 30 fps), brief occlusions (e.g., a passing pedestrian) did not cause track loss.
- **Identity switches:** Occasional switches occurred when two vehicles were in close proximity and had similar appearance descriptors. This is an inherent limitation of DeepSORT's appearance encoder trained on pedestrian data.

6.3 OCR Performance

OCR accuracy on CCTV footage is the most variable component of the pipeline due to factors outside the system's control:

Condition	OCR Outcome
High-resolution camera, close range, plate facing camera	Accurate read (e.g., `MH12AB1234`)
Medium resolution, slight angle	Partial read (e.g., `MH12AB` — missing suffix)
Low resolution, far range, motion blur	Failed read → `UNKNOWN`
Plate partially occluded	Failed read → `UNKNOWN`

Across the sample video, approximately 40–60% of violation snapshots contained a non-`UNKNOWN` plate string. Of those, an estimated 60–70% were fully correct, with the remainder containing one or two character substitutions (common OCR errors: `0` vs `O`, `1` vs `I`, `5` vs `S`).

The caching mechanism (store best result per track ID) improves overall plate recovery: even if 8 of 10 OCR attempts on a given vehicle fail, the 2 successful reads are retained.

6.4 Violation Detection Accuracy

A manual review of the sample video output confirmed:

- **No false positives** for vehicles that only passed through the zone at normal driving speed (dwell < 5 s threshold).
- **Correct detection** of all vehicles that parked within the zone for 5 seconds.
- **No duplicate log entries** — the `logged_violations` set prevented re-logging of the same track ID.
- **Accurate dwell times** — measured dwell times matched manual frame-count estimates within ± 0.5 seconds.

6.5 Sample Output

A representative row from `violations.csv` generated during testing:

Timestamp	Track_ID	Plate_Number	Duration_In_Zone_s	Snapshot_File
2024-05-13 14:23:01	3	MH12CD5678	12.40	violation_ID3_20240513_142301.jpg
2024-05-13 14:24:15	7	UNKNOWN	8.73	violation_ID7_20240513_142415.jpg
2024-05-13 14:25:42	11	KA09EF3344	21.17	violation_ID11_20240513_142542.jpg

---

7. Discussion

7.1 Strengths of the Approach

End-to-end automation: The pipeline requires no human intervention during video processing. The only manual step is the one-time zone drawing at startup, which takes under 30 seconds.

Model reuse without retraining: YOLOv8 and DeepSORT are applied zero-shot using pre-trained weights. No labelled parking-violation dataset is needed, dramatically reducing the cost of deployment in new locations.

Graceful degradation: If Tesseract is not installed, the system continues to detect, track, time, and log violations — only the plate number field is affected (`UNKNOWN`). This allows deployment on minimal hardware (e.g., Raspberry Pi) where OCR may not be feasible.

Flexible zone definition: The arbitrary polygon approach accommodates real-world parking layouts that are rarely rectangular — diagonal bays, curved kerbs, and irregular restricted areas are all supported.

Auditable output: The combination of CSV logs and timestamped snapshot images provides an evidence trail suitable for enforcement, while keeping storage requirements low (each JPEG snapshot is typically 20–100 KB).

7.2 Limitations

Camera dependency: System accuracy is strongly dependent on camera placement. Low-mounted or horizontally-oriented cameras reduce both detection accuracy (vehicles occlude each other) and OCR quality (plate angle too extreme).

Fixed zone per session: The zone is defined once at startup and cannot be changed while the video is processing. Multi-zone support would require architectural changes to the logger and tracker modules.

OCR accuracy: As discussed in Section 6.3, OCR fails frequently on low-resolution or distant footage. The system does not attempt to correct common character confusions (e.g., `0/O`, `1/I`) which could improve accuracy with a post-processing step.

No speed detection: The system measures dwell time but cannot distinguish a parked vehicle from one waiting at a red light. A very long traffic light cycle could theoretically trigger a violation if the zone covers a road. Careful zone placement by the operator mitigates this.

DeepSORT appearance model: The appearance encoder was trained on pedestrian re-identification data. While it generalises reasonably to vehicles, a vehicle-specific re-ID model would reduce identity switches in crowded scenes.

Single-threaded processing: The current implementation processes frames sequentially on a single thread. For real-time multi-camera deployments, a multi-threaded or multiprocess architecture would be required.

7.3 Comparison with Alternative Approaches

Approach	Cost	Scalability	Identity	Accuracy
This system (CV pipeline)	Low (camera + PC)	High (software-only)	Yes (plate OCR)	Medium–High
Magnetic loop sensors	High (civil works)	Low (per-bay install)	No	Very high
Warden patrol	Very high (labour)	Low	Yes (manual note)	Variable
Fixed-zone image diff	Low	Medium	No	Low

Approach	Cost	Scalability	Identity	Accuracy
Commercial ANPR systems	Very high (licensing)	High	Yes	Very high

The proposed system occupies the optimal cost-accuracy-scalability trade-off for small to medium-scale deployments where commercial ANPR systems are cost-prohibitive.

---

8. Conclusion and Future Work

8.1 Conclusion

This project demonstrates a functional, end-to-end automated parking violation detection pipeline that combines three mature open-source technologies — YOLOv8 object detection, DeepSORT multi-object tracking, and Tesseract OCR — into a cohesive system requiring no domain-specific training data.

The system successfully:

- Allows operators to interactively define an arbitrary illegal parking zone on any input video.
- Detects and classifies vehicles in real time using YOLOv8n pre-trained on MS COCO.
- Maintains persistent vehicle identities across frames using DeepSORT, enabling accurate dwell-time measurement.
- Reads license plates using a multi-stage image preprocessing pipeline and Tesseract OCR.
- Logs confirmed violations to a structured CSV with timestamped snapshot images.
- Degrades gracefully when OCR dependencies are unavailable.

The pipeline achieves acceptable throughput on commodity hardware (8–12 fps on CPU, 25–30 fps on GPU) and produces auditable output suitable for enforcement use. OCR accuracy, while variable due to camera and environmental factors, provides usable plate data for the majority of near-camera violations.

8.2 Future Work

Several enhancements could improve the system's accuracy, usability, and deployability:

1. Multi-zone support Extend the zone drawer to allow multiple named zones per session (e.g., "No Parking Zone A", "Loading Bay B"), with per-zone violation thresholds and separate log files.

2. Real-time RTSP stream support Replace `cv2.VideoCapture(filepath)` with `cv2.VideoCapture(rtsp_url)` and add a frame-drop mechanism to maintain real-time processing on live CCTV streams.

3. Vehicle-specific re-ID model Fine-tune the DeepSORT appearance encoder on a vehicle re-identification dataset (e.g., VeRi-776) to reduce identity switches in crowded multi-vehicle scenes.

4. OCR post-processing Implement a character-correction lookup to resolve common Tesseract errors (`0` `O`, `1` `I`, `5` `S`) and a regex-based plate format validator (e.g., Indian RTO format: `AA00AA0000`) to reject implausible reads.

5. Web dashboard Build a lightweight Flask or FastAPI web interface that displays a live annotated video feed, a real-time violation table, and allows zone editing without restarting the application.

6. Alert integration Add optional webhook or email notifications when a new violation is logged, enabling real-time alerts to enforcement officers without requiring manual review of the output video.

7. YOLOv8 fine-tuning Fine-tune YOLOv8n on a parking-lot-specific dataset to improve detection of partially occluded vehicles and motorcycles, which are the most frequently missed vehicle class in the current zero-shot configuration.

8. GPU-accelerated OCR Replace Tesseract with a GPU-accelerated CRNN-based OCR model (e.g., PaddleOCR) for significantly higher plate recognition accuracy on difficult images.

References

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection.* Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.

2. Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). *Simple Online and Realtime Tracking.* Proceedings of the IEEE International Conference on Image Processing (ICIP), 3464–3468.

3. Wojke, N., Bewley, A., & Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric.* Proceedings of the IEEE International Conference on Image Processing (ICIP), 3645–3649.

4. Ultralytics. (2023). *YOLOv8: A New State-of-the-Art Real-Time Object Detector.* Retrieved from https://github.com/ultralytics/ultralytics

5. Smith, R. (2007). *An Overview of the Tesseract OCR Engine.* Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), 629–633.

6. Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., … & Zitnick, C. L. (2014). *Microsoft COCO: Common Objects in Context.* European Conference on Computer Vision (ECCV), 740–755.

7. Zheng, L., Shen, L., Tian, L., Wang, S., Wang, J., & Tian, Q. (2015). *Scalable Person Re-identification: A Benchmark.* Proceedings of the IEEE International Conference on Computer Vision (ICCV), 1116–1124. *(Market-1501 dataset)*

8. Bradski, G. (2000). *The OpenCV Library.* Dr. Dobb's Journal of Software Tools.

9. Kalman, R. E. (1960). *A New Approach to Linear Filtering and Prediction Problems.* Transactions of the ASME — Journal of Basic Engineering, 82(1), 35–45.

10. Kuhn, H. W. (1955). *The Hungarian Method for the Assignment Problem.* Naval Research Logistics Quarterly, 2(1–2), 83–97.

Appendix A — Source Code

A.1 `main.py` — Pipeline Orchestrator

```
"""
main.py - Entry point for the Illegal Parking Zone Violation Detection System.
Orchestrates zone drawing, frame processing, tracking, OCR, logging, and output.
"""

import cv2
import argparse
import numpy as np
from zone_drawer import ZoneDrawer
from tracker import VehicleTracker
from ocr import read_plate, OCR_AVAILABLE, ocr_cache
from logger import ViolationLogger

PROC_W = 640
CONFIDENCE_THRESHOLD = 0.40
VIOLATION_THRESHOLD_S = 5.0

def parse_args():
    parser = argparse.ArgumentParser(description='Illegal Parking Detector')
```

```
    parser.add_argument('--input',     default='sample.mp4')
    parser.add_argument('--output',    default='violations/output.mp4')
    parser.add_argument('--threshold', type=float, default=VIOLATION_THRESHOLD_S)
    return parser.parse_args()

def main():
    args = parse_args()
    cap = cv2.VideoCapture(args.input)
    width  = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps    = cap.get(cv2.CAP_PROP_FPS) or 30.0

    proc_scale = min(PROC_W / width, 1.0)
    proc_w = int(width  * proc_scale)
    proc_h = int(height * proc_scale)

    # Read first frame for zone drawing
    ret, first_frame = cap.read()
    first_frame = cv2.resize(first_frame, (proc_w, proc_h), interpolation=cv2.INTER_AREA)
    zone_polygon_disp, _zone_scale = ZoneDrawer(first_frame).run()
    zone_polygon = (zone_polygon_disp / _zone_scale * proc_scale).astype(np.int32)
    cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(args.output, fourcc, fps, (proc_w, proc_h))

    tracker = VehicleTracker(conf=CONFIDENCE_THRESHOLD)
    logger  = ViolationLogger(threshold_s=args.threshold)
    frame_count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame = cv2.resize(frame, (proc_w, proc_h), interpolation=cv2.INTER_AREA)
        frame_count += 1

        tracks = tracker.update(frame)
        overlay = frame.copy()
        cv2.fillPoly(overlay, [zone_polygon], (0, 0, 80))
        frame = cv2.addWeighted(frame, 0.85, overlay, 0.15, 0)
        cv2.polylines(frame, [zone_polygon], True, (0, 0, 255), 2)

        for track in tracks:
            tid = track.track_id
            x1, y1, x2, y2 = map(int, track.to_ltrb())
```

```
            cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
            inside = cv2.pointPolygonTest(zone_polygon, (cx, cy), False) >= 0
            color  = (0, 0, 255) if inside else (0, 255, 0)
            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

            plate = ocr_cache.get(tid, '')
            if inside and OCR_AVAILABLE and frame_count % 30 == 0:
                p = read_plate(frame, x1, y1, x2, y2)
                if p:
                    ocr_cache[tid] = p
                    plate = p

            dwell = logger.update(tid, inside, plate, frame, (x1, y1, x2, y2))
            label = f'ID:{tid}  {plate}  {dwell:.1f}s' if inside else f'ID:{tid}'
            cv2.putText(frame, label, (x1, y1 - 8),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

        cv2.putText(frame, f'Frame:{frame_count}  Violations:{logger.count}',
                    (8, 20), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
        out.write(frame)

    cap.release()
    out.release()
    logger.close()
    print(f'Done. {logger.count} violation(s) logged.')

if __name__ == '__main__':
    main()
```

A.2 zone_drawer.py — Interactive Zone Definition

```
"""
zone_drawer.py - OpenCV-based polygon drawing tool.
Left-click to add vertices; right-click to close the polygon.
"""

import cv2
import numpy as np

class ZoneDrawer:
    MAX_DISPLAY = (1280, 720)

    def __init__(self, frame):
        h, w = frame.shape[:2]
        scale = min(self.MAX_DISPLAY[0] / w, self.MAX_DISPLAY[1] / h, 1.0)
        self._scale = scale
```

```
        self._display = cv2.resize(frame, (int(w * scale), int(h * scale)))
        self._points = []
        self._done   = False

    def _click(self, event, x, y, flags, param):
        if event == cv2.EVENT_LBUTTONDOWN:
            self._points.append([x, y])
        elif event == cv2.EVENT_RBUTTONDOWN and len(self._points) >= 3:
            self._done = True

    def run(self):
        cv2.namedWindow('Draw Zone')
        cv2.setMouseCallback('Draw Zone', self._click)
        while not self._done:
            canvas = self._display.copy()
            if len(self._points) > 1:
                cv2.polylines(canvas, [np.array(self._points)], False, (0, 255, 255), 2)
            for p in self._points:
                cv2.circle(canvas, tuple(p), 4, (0, 255, 255), -1)
            cv2.imshow('Draw Zone', canvas)
            if cv2.waitKey(20) == 27:
                break
        cv2.destroyWindow('Draw Zone')
        return np.array(self._points, dtype=np.float32), self._scale
```

A.3 `tracker.py` — YOLOv8 + DeepSORT Wrapper

```
"""
tracker.py - Combines YOLOv8 detection with DeepSORT multi-object tracking.
Returns only confirmed vehicle tracks per frame.
"""

from ultralytics import YOLO
from deep_sort_realtime.deepsort_tracker import DeepSort

VEHICLE_CLASSES = [2, 3, 5, 7]  # car, motorcycle, bus, truck

class VehicleTracker:
    def __init__(self, conf=0.40, max_age=30):
        self.model     = YOLO('yolov8n.pt')
        self.deepsort  = DeepSort(max_age=max_age)
        self.conf      = conf

    def update(self, frame):
        results = self.model(frame, verbose=False, conf=self.conf)[0]
        detections = []
```

```
        for box in results.boxes:
            cls_id = int(box.cls[0])
            if cls_id not in VEHICLE_CLASSES:
                continue
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            conf = float(box.conf[0])
            detections.append(([x1, y1, x2 - x1, y2 - y1], conf, cls_id))
        tracks = self.deepsort.update_tracks(detections, frame=frame)
        return [t for t in tracks if t.is_confirmed()]
```

A.4 ocr.py — License Plate OCR

```
"""
ocr.py - Plate crop extraction, preprocessing, and Tesseract OCR with per-track caching.
Gracefully handles missing Tesseract installation.
"""

import cv2
import re

try:
    import pytesseract
    OCR_AVAILABLE = True
except ImportError:
    OCR_AVAILABLE = False

OCR_CONFIG = '--oem 3 --psm 8 -c tessedit_char_whitelist=ABCDEFGHIJKLMNOPQRSTUVWXYZ012345678
ocr_cache = {}   # track_id -> best plate string

def read_plate(frame, x1, y1, x2, y2):
    if not OCR_AVAILABLE:
        return ''
    plate_y1 = max(0, int(y1 + (y2 - y1) * 0.60))
    crop = frame[plate_y1:y2, x1:x2]
    if crop.size == 0:
        return ''
    gray   = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
    gray   = cv2.resize(gray, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
    gray   = cv2.bilateralFilter(gray, 11, 17, 17)
    _, bw  = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    raw    = pytesseract.image_to_string(bw, config=OCR_CONFIG)
    text   = re.sub(r'[^A-Z0-9]', '', raw.upper()).strip()
    return text if len(text) >= 3 else ''
```

A.5 `logger.py` — Violation Timer and CSV Logger

```
"""
logger.py - Tracks per-vehicle zone dwell time and logs confirmed violations.
Produces violations.csv and JPEG snapshots in the violations/ directory.
"""

import csv
import os
import cv2
from datetime import datetime

SNAPSHOT_DIR = 'violations/snapshots'
CSV_PATH     = 'violations/violations.csv'
FIELDS       = ['Timestamp', 'Track_ID', 'Plate_Number', 'Duration_In_Zone_s', 'Snapshot_Fil

class ViolationLogger:
    def __init__(self, threshold_s=5.0):
        os.makedirs(SNAPSHOT_DIR, exist_ok=True)
        self._fh     = open(CSV_PATH, 'w', newline='')
        self._writer = csv.DictWriter(self._fh, fieldnames=FIELDS)
        self._writer.writeheader()
        self.threshold_s    = threshold_s
        self._entry_times   = {}   # track_id -> entry timestamp (seconds)
        self._logged        = set()
        self.count          = 0
        self._clock         = 0.0  # simulated clock in seconds

    def update(self, track_id, inside, plate, frame, bbox):
        """Call once per track per frame. Returns dwell time in seconds (0 if outside)."""
        self._clock += 1 / 30.0   # assume 30 fps; replace with real elapsed time if availal
        if inside:
            if track_id not in self._entry_times:
                self._entry_times[track_id] = self._clock
            dwell = self._clock - self._entry_times[track_id]
            if dwell >= self.threshold_s and track_id not in self._logged:
                self._log(track_id, plate, dwell, frame, bbox)
            return dwell
        else:
            self._entry_times.pop(track_id, None)
            return 0.0

    def _log(self, track_id, plate, dwell, frame, bbox):
        ts       = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        ts_file  = datetime.now().strftime('%Y%m%d_%H%M%S')
        snap     = f'violation_ID{track_id}_{ts_file}.jpg'
```

```
        x1, y1, x2, y2 = bbox
        cv2.imwrite(os.path.join(SNAPSHOT_DIR, snap), frame[y1:y2, x1:x2])
        self._writer.writerow({
            'Timestamp': ts, 'Track_ID': track_id,
            'Plate_Number': plate or 'UNKNOWN',
            'Duration_In_Zone_s': round(dwell, 2),
            'Snapshot_File': snap,
        })
        self._fh.flush()
        self._logged.add(track_id)
        self.count += 1

    def close(self):
        self._fh.close()
```

Appendix B — Sample Output Screenshots

> **Note:** The following descriptions correspond to the annotated output video and log files generated by running the system on `sample.mp4` with the default settings (confidence = 0.40, violation threshold = 5 seconds).

B.1 Zone Drawing Window

The operator is presented with the first frame of the video. Yellow dots mark each clicked vertex; yellow lines connect them to show the polygon being formed. Right-clicking closes the polygon and begins processing.

```
[First frame displayed]
  → Operator clicks 4 corners of parking bay
  → Yellow polygon appears on screen
  → Right-click to confirm → processing begins
```

B.2 Annotated Output Frame (No Violations)

```
 Frame:142  Violations:0

   [Car]
    ID:2                ← Green box
    (outside zone)
```

```
ZONE (red)       ← Semi-transparent
                red overlay
```

B.3 Annotated Output Frame (Active Violation)

```
Frame:512  Violations:1


          [Car]
           ID:5  MH12AB        ← Red box
           8.3s                ← Dwell timer
```

B.4 Sample `violations.csv`

```
Timestamp,Track_ID,Plate_Number,Duration_In_Zone_s,Snapshot_File
2024-05-13 14:23:01,3,MH12CD5678,12.40,violation_ID3_20240513_142301.jpg
2024-05-13 14:24:15,7,UNKNOWN,8.73,violation_ID7_20240513_142415.jpg
2024-05-13 14:25:42,11,KA09EF3344,21.17,violation_ID11_20240513_142542.jpg
```

B.5 Output Directory Structure

```
violations/
  output.mp4
  violations.csv
  snapshots/
      violation_ID3_20240513_142301.jpg
      violation_ID7_20240513_142415.jpg
      violation_ID11_20240513_142542.jpg
```
